

ui.plotly 全面详解

`ui.plotly` 是 NiceGUI 框架中深度集成 Plotly 可视化库的核心组件，专为 Web 环境下的交互式图表设计。Plotly 以动态交互、丰富图表类型（含 2D/3D 图表、地理信息图等）和高颜值可视化效果著称，`ui.plotly` 则通过简洁的 Python API 封装，实现 Plotly 图表与 Web 应用的无缝融合，支持动态更新、事件监听、样式定制等高级功能。以下从核心特性、使用场景、详细配置、高级技巧等维度展开全面解析。

一、核心概述

1. 本质与定位

- 封装 Plotly 的 `Figure` 对象，无需编写 JavaScript，通过 Python 原生语法即可创建交互式 Web 图表。
- 支持 Plotly 全量图表类型（折线图、柱状图、散点图、3D 图表、热力图、地图等），兼容 Plotly Express 和 Plotly Graph Objects (go) 两种 API。
- 与 NiceGUI 生态深度集成，支持组件样式定制、事件监听、可见性绑定、动态数据更新，适配从简单报表到复杂可视化的全场景需求。

2. 核心参数（初始化时必填 / 常用）

参数名	类型	说明
<code>figure</code>	<code>plotly.graph_objects.Figure</code>	Plotly 图表对象（必填，通过 <code>plotly.express</code> 或 <code>plotly.graph_objects</code> 创建）。
<code>config</code>	<code>dict</code>	Plotly 图表配置字典，控制交互行为（如是否显示工具栏、缩放模式、图例位置等），默认使用 Plotly 全局配置。
<code>style</code>	<code>Style[Self]</code>	图表容器样式，支持 Tailwind CSS（如 <code>bg-gray-50</code> 、 <code>rounded-lg</code> ）。
<code>classes</code>	<code>Classes[Self]</code>	图表容器的 HTML 类名，用于自定义样式复用。
<code>visible</code>	<code>bool</code>	图表初始可见性（默认 <code>True</code> ），支持后续动态修改或绑定。
<code>html_id</code>	<code>str</code>	图表在 HTML DOM 中的唯一 ID（v2.16.0+ 支持），用于自定义 JS 交互。

3. 核心优势

- 强交互性**：支持缩放、平移、悬停显示详情、图例切换、数据下载（PNG/SVG/CSV）等原生交互功能。
- 丰富图表类型**：覆盖统计图表、科学可视化、地理信息图、3D 图表等，满足多样化需求。
- 动态更新高效**：修改 `Figure` 对象后调用 `update()` 即可刷新图表，支持批量数据更新与实时监控。
- 高颜值默认样式**：Plotly 自带现代化设计风格，无需额外定制即可适配 Web 界面。

二、基础使用场景与示例

1. 基础静态图表（Plotly Express 用法）

Plotly Express (px) 是 Plotly 简化版 API，通过一行代码即可创建复杂图表，适用于快速可视化场景。

示例：基础折线图（Plotly Express）

```
from nicegui import ui
import plotly.express as px
import pandas as pd

# 1. 准备数据（Pandas DataFrame 或字典均可）
df = pd.DataFrame({
    'x': [1, 2, 3, 4, 5],
    'y': [10, 20, 15, 25, 30],
    'category': ['A', 'A', 'B', 'B', 'A']
})

# 2. 创建 Plotly Figure 对象
fig = px.line(
    df,
    x='x',
    y='y',
    color='category', # 按类别着色
    title='Basic Line Chart (Plotly Express)',
    labels={'x': 'X Axis', 'y': 'Y Axis'},
    markers=True # 显示数据点标记
)

# 3. 嵌入 NiceGUI 页面
ui.plotly(fig)

ui.run()
```

2. 复杂定制图表（Plotly Graph Objects 用法）

Plotly Graph Objects (go) 是 Plotly 底层 API，支持精细化配置图表的每一个元素（如坐标轴样式、图例位置、数据系列属性等）。

示例：定制化柱状图（Plotly Graph Objects）

```
from nicegui import ui
import plotly.graph_objects as go

# 1. 创建 Figure 对象
fig = go.Figure()

# 2. 添加数据系列（柱状图）
fig.add_trace(go.Bar(
    x=['Jan', 'Feb', 'Mar', 'Apr'],
    y=[120, 180, 150, 220],
```

```

        name='Sales',
        marker_color='#28738a', # 柱子颜色
        marker_line_width=1,     # 柱子边框宽度
        marker_line_color='black' # 柱子边框颜色
    ))

# 3. 定制图表样式
fig.update_layout(
    title='Monthly Sales (Customized)',
    title_font=dict(size=16, weight='bold'),
    xaxis_title='Month',
    yaxis_title='Sales Amount',
    yaxis_range=[0, 250], # 固定 Y 轴范围
    legend=dict(loc='upper right', font_size=12),
    plot_bgcolor='rgba(245, 245, 245, 0.8)', # 图表背景色
    paper_bgcolor='transparent' # 画布背景透明（适配容器样式）
)

# 4. 嵌入 NiceGUI, 添加容器样式
ui.plotly(
    fig,
    style='bg-gray-50 rounded-xl shadow-md p-4' # Tailwind 样式
)

ui.run()

```

3. 动态更新图表数据

修改 Plotly Figure 对象的数据源（如 `fig.data[0].y`），调用 `ui.plotly.update()` 方法触发图表刷新，适用于实时数据监控、交互控制场景。

示例：点击按钮切换数据系列

```

from nicegui import ui
import plotly.graph_objects as go
import numpy as np

# 1. 初始化图表（正弦曲线）
x = np.linspace(0, 10, 100)
fig = go.Figure(go.Scatter(
    x=x,
    y=np.sin(x),
    name='sin(x)',
    mode='lines+markers', # 线条+标记点
    line_color='#b687ac'
))

fig.update_layout(
    title='Dynamic Data Update',
    yaxis_range=[-1.5, 1.5]
)

# 2. 嵌入图表并保存组件引用

```

```

plotly_comp = ui.plotly(fig)

# 3. 定义更新函数（切换 sin(x)/cos(x)）
def toggle_data():
    current_name = fig.data[0].name
    if current_name == 'sin(x)':
        fig.data[0].y = np.cos(x)
        fig.data[0].name = 'cos(x)'
        fig.data[0].line_color = '#28738a'
    else:
        fig.data[0].y = np.sin(x)
        fig.data[0].name = 'sin(x)'
        fig.data[0].line_color = '#b687ac'
    # 更新图例
    fig.update_layout(legend_title_text='Function')
    # 触发图表刷新
    plotly_comp.update()

# 添加控制按钮
ui.button('Toggle sin/cos', on_click=toggle_data)

ui.run()

```

4. 3D 图表渲染

Plotly 原生支持 3D 图表（如 3D 散点图、曲面图、折线图），`ui.plotly` 自动适配 3D 渲染逻辑，无需额外配置。

示例：3D 曲面图

```

from nicegui import ui
import plotly.graph_objects as go
import numpy as np

# 生成 3D 数据（网格）
x = np.linspace(-5, 5, 50)
y = np.linspace(-5, 5, 50)
x_grid, y_grid = np.meshgrid(x, y)
z_grid = np.sin(np.sqrt(x_grid**2 + y_grid**2)) # 正弦曲面

# 创建 3D 曲面图
fig = go.Figure(go.Surface(
    x=x_grid,
    y=y_grid,
    z=z_grid,
    colorscale='viridis', # 颜色映射
    colorbar_title='Z value'
))

fig.update_layout(
    title='3D Surface Plot',
    scene=dict(
        xaxis_title='X',
        yaxis_title='Y',

```

```

        zaxis_title='z',
        camera=dict(eye=dict(x=1.5, y=1.5, z=0.5)) # 相机视角
    ),
    width=800,
    height=600
)

ui.plotly(fig)

ui.run()

```

5. 地理信息图 (Map)

Plotly 支持多种地理图表（如散点地图、热力地图、边界地图），适用于数据可视化与地理分析场景。

示例：世界地图散点图

```

from nicegui import ui
import plotly.express as px
import pandas as pd

# 准备数据（国家、纬度、经度、数值）
df = pd.DataFrame({
    'country': ['China', 'USA', 'India', 'Brazil', 'Nigeria'],
    'lat': [35.8617, 37.0902, 20.5937, -14.235, 9.082],
    'lon': [104.1954, -95.7129, 78.9629, -51.9253, 8.6753],
    'value': [1400, 330, 1380, 210, 200]
})

# 创建地理散点图
fig = px.scatter_geo(
    df,
    lat='lat',
    lon='lon',
    size='value', # 点大小由数值决定
    size_max=50, # 最大点大小
    color='value', # 颜色由数值决定
    hover_name='country', # 悬停显示国家名
    title='World Population (Scatter Map)',
    projection='natural earth' # 地图投影方式
)

ui.plotly(fig, style='width: 100%; height: 600px')

ui.run()

```

6. 事件监听（点击 / 悬停交互）

通过 `on` 方法绑定 Plotly 原生事件（如点击数据点、悬停、图例切换），实现自定义交互逻辑（如显示详情、跳转页面）。

示例：点击数据点显示详情

```

from nicegui import ui
import plotly.graph_objects as go
import pandas as pd

# 准备数据
df = pd.DataFrame({
    'product': ['A', 'B', 'C', 'D'],
    'sales': [120, 180, 150, 220],
    'profit': [30, 45, 35, 55]
})

# 创建柱状图
fig = go.Figure(go.Bar(
    x=df['product'],
    y=df['sales'],
    name='Sales',
    hover_data={'profit': True} # 悬停显示利润数据
))

fig.update_layout(title='Product Sales & Profit')

# 嵌入图表并添加状态标签
plotly_comp = ui.plotly(fig)
detail_label = ui.label('Click a bar to view details')

# 绑定点击事件（事件类型: plotly_click）
def on_bar_click(e):
    # e.args 包含事件详情: points 数组（选中的数据点）
    points = e.args['points']
    if points:
        point = points[0]
        product = point['x']
        sales = point['y']
        profit = point['customdata'][0] # 获取 hover_data 中的利润
        detail_label.set_text(f'Product: {product} | Sales: {sales} | Profit: {profit}')

plotly_comp.on('plotly_click', on_bar_click)

ui.run()

```

7. 定制图表交互配置

通过 `config` 参数控制 Plotly 图表的交互行为（如是否显示工具栏、缩放模式、是否允许下载等）。

示例：禁用工具栏与缩放功能

```

from nicegui import ui
import plotly.express as px
import pandas as pd

df = pd.DataFrame({
    'x': [1, 2, 3, 4],

```

```

        'y': [10, 20, 15, 25]
    })

fig = px.line(df, x='x', y='y', title='Static Chart (No Interaction)')

# 配置：禁用工具栏、禁用缩放/平移、禁用数据下载
config = {
    'displayModeBar': False, # 隐藏工具栏
    'staticPlot': True,      # 静态图表，禁用缩放/平移
    'toImageButtonOptions': {'display': False} # 隐藏下载按钮（若显示工具栏）
}

ui.plotly(fig, config=config)

ui.run()

```

8. 实时数据流监控（定时器更新）

结合 `ui.timer` 实现定时更新图表数据，适用于实时监控场景（如传感器数据、系统性能指标）。

示例：实时动态折线图

```

from nicegui import ui
import plotly.graph_objects as go
import numpy as np
from collections import deque

# 初始化：缓存最近 50 个数据点
max_data_points = 50
x_data = deque(range(max_data_points), maxlen=max_data_points)
y_data = deque(np.random.randn(max_data_points), maxlen=max_data_points)

# 创建图表
fig = go.Figure(go.Scatter(
    x=list(x_data),
    y=list(y_data),
    mode='lines',
    line_color='#e74c3c'
))

fig.update_layout(
    title='Real-Time Data Monitor',
    xaxis_title='Time',
    yaxis_title='Value',
    yaxis_range=[-3, 3],
    xaxis=dict(range=[0, max_data_points])
)

plotly_comp = ui.plotly(fig)

# 定时器：每 200ms 更新一次数据
def update_realtime_data():
    # 添加新数据点

```

```
new_y = np.random.randn()
y_data.append(new_y)
x_data.append(x_data[-1] + 1)
# 更新图表数据
fig.data[0].x = list(x_data)
fig.data[0].y = list(y_data)
# 更新 X 轴范围（自适应）
fig.update_layout(xaxis=dict(range=[x_data[0], x_data[-1]]))
# 触发刷新
plotly_comp.update()

ui.timer(0.2, update_realtime_data)

ui.run()
```

三、核心属性与方法详解

1. 常用属性

属性名	类型	说明
<code>figure</code>	<code>plotly.graph_objects.Figure</code>	可读可写，关联的 Plotly Figure 对象（修改后需调用 <code>update()</code> 刷新）。
<code>config</code>	<code>dict</code>	可读可写，图表交互配置（修改后需调用 <code>update()</code> 生效）。
<code>visible</code>	<code>BindableProperty</code>	可读可写，控制图表是否可见（支持绑定，如 <code>bind_visibility()</code> ）。
<code>style</code>	<code>Style[Self]</code>	可读可写，图表容器样式，支持动态修改（如 <code>plotly_comp.style('bg-blue-50')</code> ）。
<code>classes</code>	<code>Classes[Self]</code>	可读可写，图表容器的 HTML 类名，用于样式复用。
<code>html_id</code>	<code>str</code>	图表在 HTML DOM 中的唯一 ID（v2.16.0+ 支持）。

2. 核心方法

方法名	作用	参数说明
<code>update()</code>	手动触发图表刷新，修改 <code>figure</code> 或 <code>config</code> 后必须调用此方法。	-
<code>on(type, handler)</code>	绑定 Plotly 原生事件或 Web 标准事件（如 <code>plotly_click</code> 、 <code>click</code> ）。	- <code>type</code> ：事件类型（Plotly 事件需前缀 <code>plotly_</code> ，如 <code>plotly_hover</code> ）； - <code>handler</code> ：回调函数，接收事件参数 <code>e</code> （ <code>e.args</code> 包含事件详情）。

方法名	作用	参数说明
<code>bind_visibility(target)</code>	双向绑定图表可见性到目标对象的属性（如开关的 <code>value</code> 属性）。	<code>target_object</code> : 目标对象; <code>target_name</code> : 属性名（默认 <code>visible</code> ）。
<code>set_style(**kwargs)</code>	设置图表容器样式（等价于 <code>style</code> 属性），支持 Tailwind CSS。	关键字参数如 <code>bg_gray_50</code> 、 <code>rounded-lg</code> 等。
<code>delete()</code>	删除图表组件及关联的 Plotly Figure 对象，释放内存。	-

3. 常用 Plotly 事件类型

事件类型	触发场景
<code>plotly_click</code>	点击图表数据点时触发（如柱状图、折线图的标记点）。
<code>plotly_hover</code>	鼠标悬停在数据点上时触发。
<code>plotly_unhover</code>	鼠标离开数据点时触发。
<code>plotly_legendclick</code>	点击图例时触发（可用于控制数据系列显示 / 隐藏）。
<code>plotly_relayout</code>	图表布局变化时触发（如缩放、平移、调整窗口大小）。

四、高级技巧与注意事项

1. 性能优化

- 数据量控制**：实时场景建议限制数据点数量（如 `max_data_points=100`），避免数据过多导致图表渲染卡顿。
- 批量更新**：高频数据更新时，可累计多次数据后集中调用 `update()`（如每 5 次数据推送更新一次图表）。
- 图表类型选择**：大数据量场景优先使用 `scattergl`（WebGL 加速散点图）而非普通 `scatter`，提升渲染性能：

```
fig.add_trace(go.Scattergl(x=x, y=y, mode='markers')) # WebGL 加速
```

2. 样式定制技巧

- 容器样式**：通过 Tailwind CSS 实现响应式布局（如 `style='width: 100%; max-width: 1200px'`），适配不同屏幕尺寸。
- 图表内部样式**：通过 `fig.update_layout()` 定制标题、坐标轴、图例、背景色等；通过 `fig.update_traces()` 批量修改数据系列样式：

```
# 批量修改所有数据系列的线条宽度
fig.update_traces(line_width=2)
```

- **主题复用**：使用 Plotly 内置主题（如 `plotly_dark`、`seaborn`）或自定义主题，统一图表风格：

```
fig.update_layout(template='plotly_dark') # 深色主题
```

3. 动态更新最佳实践

- **修改现有数据系列**：避免频繁调用 `fig.add_trace()` 或 `fig.remove_trace()`，优先修改现有系列的 `x/y` 数据（如 `fig.data[0].y = new_y`）。
- **批量修改布局**：多次布局调整后集中调用 `fig.update_layout()`，减少刷新次数。
- **避免重复创建 Figure**：动态更新场景复用同一个 `Figure` 对象，而非每次更新都创建新对象。

4. 兼容性与版本说明

- **依赖版本**：要求 Plotly $\geq 5.0.0$ ，NiceGUI $\geq 2.0.0$ （`html_id` 需 `v2.16.0+`）。
- **浏览器兼容性**：Plotly 基于 WebGL 和 SVG 渲染，现代浏览器（Chrome、Firefox、Edge）均支持，老旧浏览器（如 IE）可能存在兼容性问题。
- **3D 图表支持**：3D 图表依赖 WebGL，部分低配置设备或浏览器可能无法正常渲染。

5. 调试技巧

- **查看 Figure 配置**：打印 `plotly_comp.figure` 可获取当前图表的完整配置，排查样式或数据问题。
- **事件参数调试**：在回调函数中打印 `e.args`，查看事件的完整详情（如点击点的坐标、数据值、系列名称）。
- **图表不显示排查**：
 1. 检查 `figure` 是否正确创建（可通过 `fig.show()` 在本地测试）。
 2. 确认修改 `figure` 后调用了 `update()` 方法。
 3. 检查容器样式是否设置了 `display: none` 或 `width: 0`。
 4. 切换浏览器或清除缓存，排除前端渲染问题。

五、与其他 NiceGUI 图表组件的对比

NiceGUI 提供多种图表组件，选择时需根据场景匹配核心需求，以下是 `ui.plotly` 与其他组件的关键区别：

组件	核心优势	适用场景	交互性	学习成本
<code>ui.plotly</code>	强交互、丰富图表类型、高颜值	交互式报表、实时监控、3D 可视化、地理分析	★★★★	中
<code>ui.echart</code>	兼容 ECharts 生态、支持 2D/3D 图表	传统数据可视化、复用 ECharts 配置	★★★★	中
<code>ui.matplotlib</code>	简洁上下文、兼容 Matplotlib 生态	静态图表、科学计算可视化、新开发项目	★★★★	低

组件	核心优势	适用场景	交互性	学习成本
<code>ui.pyplot</code>	兼容 Matplotlib 脚本、灵活配置	复用现有 Matplotlib 代码、复杂静态图表	***	低
<code>ui.line_plot</code>	专注实时折线图、简化数据更新	单一时序数据监控、多线条动态展示	***	低

六、总结

`ui.plotly` 是 NiceGUI 中交互性最强、功能最丰富的可视化组件，核心优势在于：

- 1. **极致交互体验**：支持缩放、平移、悬停详情、图例切换、数据下载等原生交互，无需额外开发。
- 2. **全场景覆盖**：从基础统计图表到 3D 可视化、地理信息图，满足多样化可视化需求。
- 3. **动态更新高效**：修改 `Figure` 后调用 `update()` 即可刷新，适配实时监控场景。
- 4. **样式高颜值**：默认现代化设计风格，支持深度定制，无缝融入 Web 界面。
- 5. **生态兼容**：支持 Plotly Express 和 Graph Objects 两种 API，复用现有 Plotly 代码。

适用于交互式数据报表、实时监控面板、科学计算可视化、3D 模型展示、地理信息分析等场景，尤其适合对交互体验和可视化效果有较高要求的 Web 应用。